ViennaPhysicsEngine

The manual: what the classes are, how the engine works inside, and how to build a game with it

Part I
The parts you use

→

Part II
How it works inside

→

Part III
Building a game with it

This manual explains ViennaPhysicsEngine (VPE) in plain language. You do not need to know physics engines already. You should know basic C++: classes, pointers, lambdas and `std::vector`.

Part I explains the classes you use in your code. **Part II** follows the engine step by step through one update, so you know what happens when you call it. **Part III** shows how to put the engine into a real game. New words are explained on page 3, "Words you will meet".

> **Start here if you are new**
> First read the **beginner quickstart** (physics_v2_beginner_quickstart.pdf). It shows how to build the engine and run a falling cube in eight pages. Then come back to this manual.

Every chapter ends with an **In the code** line. It tells you which file and which class or function to open in your editor. Line numbers are for the version of 12 October 2026; if the code has changed since, search for the class or function name instead.

Edition of 12 October 2026, for the main branch of ViennaPhysicsEngine. The example code in this manual uses the real engine functions, and the snippets were compiled and run with Clang 18.

Contents

Click a line to jump there.

Chapter	Page
Words you will meet	3
Part I - The parts you use	4
Part II - How the engine works inside	13
Part III - Building a game with it	24
Where to find things in the code	38

A good way to read this manual

Read the words page first. Then read Part I, especially chapters 4 and 5 about storing and removing bodies. Follow one update in Part II, chapter 10. Then try the falling body in chapter 20 and use the rest of Part III as a recipe book.

> **What was checked for this edition**
> The text was compared with the source code of 12 October 2026. The code examples use the real engine functions; the falling body, joint, collision and cloth examples were compiled and run with Clang 18. The 3D demo was built with the Microsoft compiler on Windows.

Words you will meet

Short explanations of the words this manual uses a lot.

Word	Meaning
Rigid body	A solid object that never bends, like a box. In the code: `Body`.
Cloth	A soft object made of triangles that can bend, like a flag.
World	The container that holds all bodies and cloth and moves time forward (`VPEWorld`).
Tick	One call of `world.tick(seconds)`. Your game calls it once per frame.
Time step / slot	The engine moves time in fixed small steps of 1/60 second. One tick can contain zero, one or several steps.
Inverse mass	1 divided by the mass. 0 means infinitely heavy: the body never moves.
Owner	A pointer to your own game object. The engine uses it as a name tag for the body.
Callback	A function you give to the engine. The engine calls it later, for example after a body moved.
Collision	Two objects touch or overlap.
Contact	A record of one collision, with the points where the objects touch.
Impulse	A short, strong push that changes speed at once, like a hit.
Joint / constraint	A connection between two bodies, like a hinge or a rod.
Convex	A shape without dents or holes. A box and a ball are convex; a cup is not.
Polytope	A convex shape made of flat faces, edges and corners. The cube is one.
Inertia	How hard it is to make a body spin. Like mass, but for turning.
Quaternion	Four numbers that describe a rotation. "No rotation" is `{1,0,0,0}`.
Module	A C++23 way to share code with `import` instead of `#include`.
shared_ptr	A smart pointer: the object is deleted when the last `shared_ptr` to it is gone.

Part I - The parts you use

The classes and functions your game code calls.

Click a chapter title to jump to it.

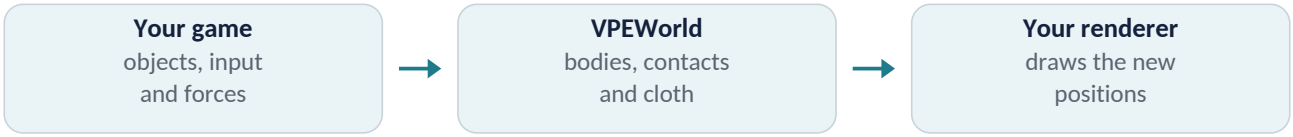
Chapter	Page
1. What the engine does for your game	5
2. The classes at a glance	6
3. Numbers, units and directions	7
4. The world and how it stores objects	8
5. Bodies and forces	9
6. Shapes and collisions	10
7. Joints	11
8. Cloth	12

Each chapter is one page. Read them in order the first time. Later you can jump straight to the chapter you need.

1. What the engine does for your game

It calculates movement. Drawing is someone else's job.

VPE calculates how solid objects and cloth move and bump into each other. Your game tells it which objects exist, where they start and which forces push them. Then the engine moves time forward. After each update it calls your callbacks, so your game can move the visible objects on screen.



You start with `import VEPhysicsEngine;`. All classes are in the namespace `vpe`. You simply create a `vpe::VPEWorld` object. There is no setup function, no window and no main loop inside the engine: your game decides when to call it.

Job	Who does it
Mass, speed, forces, collisions and joints	ViennaPhysicsEngine
Cloth movement and the new cloth corner positions	ViennaPhysicsEngine
Loading 3D models, textures and materials	Your game, or the Vienna Vulkan Engine (VVE)
Input, game rules and when to update	Your game
Drawing on the screen (graphics card)	VVE or another renderer

> **A useful picture to keep in mind**
> The **world** stores the physics state. A **Body** is the physical side of an object. Your **game object** is the owner of that body and also has a visible side. These are three different things that belong together.

In the code: modules/VPE.ixx: export module (line 98), class VPEWorld (line 237).

2. The classes at a glance

Which classes you call, and which ones only help the engine.

All classes live inside `VPEWorld`, so their full names look like `vpe::VPEWorld::Body`.

Class	What it is for
`VPEWorld`	**You use it.** Holds everything; add and remove objects; `tick` moves time forward.
`Body`, `Force`	**You use them.** A solid object, and a push that acts on it.
`Polytope`	**You use it.** A convex collision shape. The ready-made cube is `g_cube`.
`BallSocketJoint`, `HingeJoint`, `SliderJoint`, `FixedJoint`, `DistanceConstraint`	**You use them.** Connections between two bodies (chapter 7).
`Cloth`	**You use it.** A soft object made of triangles (chapter 8).
`Vertex`, `Edge`, `Face`	Helpers: the corners, edges and faces that make up a `Polytope`.
`Contact`, `ContactPoint`	Helpers: the engine's notes about a collision. You do not create them.
`SatQuery`, `EdgeQuery`, `FaceQuery`	Helpers for the exact collision test (chapter 14).
`ClothMassPoint`, `ClothTriangle`, `ClothConstraint`	Helpers: the inner parts of a cloth. `Cloth` makes them for you.
`MapWrapper`	Helper: the list in which the world stores its bodies (chapter 4).

Many member variables are `public`, so you *can* change them directly. That does not mean every change is safe. Use the functions (`addBody`, `setForce` and so on) when they exist. Chapter 24 explains what to do when you change values by hand.

In the code: modules/VPE.ixx: class VPEWorld (line 237), struct Force (line 268), class Body (line 461), class Constraint (line 1869), class Cloth (line 3340).

3. Numbers, units and directions

What positions, rotations and masses mean.

The engine has its own short names for number types. You can use them without `vpe::`:

Name	Meaning
`real`	A decimal number (normally `float`).
`glmvec3`	A 3D vector: x, y, z. Used for positions, speeds and forces.
`glmquat`	A rotation (quaternion). No rotation is `glmquat{1,0,0,0}`.
`glmmat4`	A 4x4 matrix, used to place an object in the world.
`2.5_real`	Writes a number in the engine's `real` type.

Member names end with a letter that tells you the space: **W** means world space (the whole scene), **L** means local space (relative to the body itself). For example `m_positionW` is the position in the world, and `m_orientationLW` turns local directions into world directions.

Value	Meaning
`m_positionW`, `m_linear_velocityW`	Where the body is, and how fast it moves.
`m_angular_velocityW`	How fast it spins, in radians per second.
`m_scale`	The size of the body.
`m_mass_inv`	Inverse mass. 0 means the body never moves.

Up is +y. The floor is the flat plane at y = 0. Two CMake options change the number setup: `VPE_DOUBLE_ACCURACY` uses `double` instead of `float`, and `VPE_FORCE_LEFT_HANDED` changes the coordinate system. Both are off by default. Build the engine and your game with the same options.

> **A simple rule for units**
> Use 1 unit = 1 meter, time in seconds and mass in kilograms everywhere. If your 3D models use other units, convert them once when you load them. Test with one falling cube before you build a big scene.

In the code: modules/VPE.ixx: number types (around line 110), Body members (line 461); CMakeLists.txt, options at the top.

4. The world and how it stores objects

Adding, finding and removing bodies.

The world finds a body by its **owner** pointer. The owner is usually a pointer to your game object. The engine does not know what type it is (it is a `void*`); it only uses the address as a name tag. So every body needs its **own** owner address, and that address must not change. The built-in floor uses `nullptr` as its owner.

Function	What it really does
`addBody(body)`	Puts the body into the world. It does **not** check whether the owner is already used.
`getBody(owner)`	Finds a body. It returns a pair; the body is in `.second`.
`eraseBody(body)`	Removes the body, calls its erase callback, and also removes all joints that use it.
`eraseBody(owner)`	Removes the body too, but does **not** remove its joints. Prefer the version above.
`clear()`	Removes all bodies and joints. Cloth is not removed: use `clearCloths()` for that.

Bodies are stored in a `MapWrapper`. It looks like a map, but inside it is a simple list, so finding a body walks through the list. If you look up an owner that is not there, it throws an exception instead of adding an empty entry like `std::unordered_map` would. If two bodies share one owner, finding and removing them becomes unreliable.

The world and the joints hold bodies with `shared_ptr`. But a body only keeps **plain pointers** to its world and its shape. So a body does not keep the world, its shape or your game object alive: you must do that.

> **How to remove a body safely**
> Store the body's `shared_ptr` in your game object and remove it with `eraseBody(body)`. If you decide to remove it inside a callback, only note it in a list, and remove it after `tick` has returned.

In the code: modules/VPE.ixx: MapWrapper (line 954), addBody (line 1099), clear (line 1119), eraseBody (lines 1136 and 1154).

5. Bodies and forces

The two classes you use most.

A Body has a position, rotation, speed, spin, inverse mass, bounciness, friction and a list of forces. It also stores some values the engine calculates for itself, such as matrices and inertia.

```
using W = vpe::VPEWorld;
auto body = std::make_shared<W::Body>(
    &world, "crate", &owner, &W::g_cube,   // world, name, owner, shape
    glmvec3{1,1,1}, glmvec3{0,4,0},          // size, position
    glmquat{1,0,0,0},                        // rotation
    glmvec3{0,0,0}, glmvec3{0,0,0},          // speed, spin
    1.0_real, 0.2_real, 0.8_real);           // inverse mass, bounciness, friction
```

Everything after the position is optional. If you leave it out, the body has no rotation, no speed, inverse mass 0 (it does not move!), bounciness 0.2 and friction 1. The constructor makes the shape 0.1 percent bigger as a safety margin and calculates the inertia for you.

Member	How to use it
`setForce(id, force)`	Adds a force with a number (ID). Using the same ID again replaces the old force.
`removeForce(id)`	Removes that force.
`m_on_move`	Your callback: called after the body moved. You get a time value and the body.
`m_on_erase`	Your callback: called when the body is removed, so you can remove the visible object.
`updateMatrices()`, `inertiaTensorL()`	Recalculate stored values after you changed the body by hand.
`Body::computeModel(pos, rot, scale)`	Makes the matrix that places the visible object.

A Force has four fields: `m_accelW` is an acceleration (it is not divided by the mass, so all bodies fall equally fast); `m_forceW` is a force in the world (divided by the mass); `m_forceL` and `m_positionL` are a force on a point of the body, which also makes it spin.

> **Gravity is not automatic**
> The world has the value `c_gravity` (-9.81), but a new body does not use it by itself. Add it:
> `body->setForce(0ul, W::Force{{0, world.c_gravity, 0}});`

In the code: modules/VPE.ixx: struct Force (line 268), Body constructor (line 535), setForce (line 556), computeModel (line 680).

6. Shapes and collisions

The shape you see and the shape that collides are different things.

A `Polytope` is the collision shape of a body. It is made of corners (vertices), edges and flat faces, and it knows which ones are connected. The ready-made `VPEWorld::g_cube` is a cube. Many bodies can share it: each body has its own `m_scale`, so the same cube can be a small box or a long plank.

The exact collision test works only for **convex** shapes (no dents or holes). A 3D model you load, for example a chair or a car, is usually **not** convex and cannot be used directly. Instead, you approximate it with one or more convex shapes, such as a few boxes.

Helper	What it stores
`Vertex`	One corner of the shape, and which edges and faces it belongs to.
`Edge`	Two corners and the two faces next to it.
`Face`	One flat side, its direction (normal) and its edges.
`Contact`	Two bodies that touch, and the touching points from this and the last step.
`ContactPoint`	One touching point and the pushes (impulses) used there.

The world creates contacts by itself. Your game normally learns about collisions through a collision callback (chapter 25), not by reading contacts. Contacts are kept from one step to the next, so do not remove bodies or change shapes while the engine is running.

> **Keep shapes alive and unchanged**
> The parts of a `Polytope` point to each other. If you copy it, or put it in a `std::vector` that grows later, those pointers break. Build a shape once, keep it in one place and keep it alive while bodies use it.

In the code: modules/VPE.ixx: Vertex, Edge and Face (from line 276), g_cube (line 434), Contact (line 716).

7. Joints

Connect two bodies and choose how they may still move.

Joint	What stays connected	What can still move
`BallSocketJoint`	One shared point	Turning in all directions (like a shoulder)
`HingeJoint`	One shared point and an axis	Turning around that axis (like a door)
`SliderJoint`	Rotation and sideways position	Sliding along one axis (like a drawer)
`FixedJoint`	Position and rotation	Nothing: the bodies move as one
`DistanceConstraint`	The distance between the centers	Everything else (like a rod)

```
using W = vpe::VPEWorld;
// support and door are bodies that were already added.
auto hinge = std::make_shared<W::HingeJoint>(
    support, door,
    glmvec3{0, 2, 0},     // the shared point, in world coordinates
    glmvec3{0, 1, 0});    // the hinge axis (must not be zero)
hinge->enableLimit(-0.5_real, 0.5_real);   // allowed angles, in radians
world.addConstraint(hinge);
```

Other constructors: `BallSocketJoint(a, b, point)`, `FixedJoint(a, b, point)`, `SliderJoint(a, b, point, axis)` and `DistanceConstraint(a, b, distance)`.

A hinge limit is measured from the starting angle. The minimum must be smaller than the maximum, and 0 should be between them. Hinges and sliders can also have a **motor**: `enableMotor(speed, maxForce)` drives them at a speed, and `disableMotor()` stops it. Remove a joint with `removeConstraint`.

A joint keeps its two bodies alive, because it holds them with `shared_ptr`. Joined bodies still collide with each other: there is no switch to turn that off. Place their shapes so they do not block the movement you want.

In the code: modules/VPE.ixx: Constraint (line 1869), DistanceConstraint (line 1911), BallSocketJoint (line 1981), HingeJoint (line 2082), FixedJoint (line 2425), SliderJoint (line 2506).

8. Cloth

A soft object made from a triangle mesh.

To make a cloth you give: the world, a name, an owner, a move callback, an erase callback, the corner positions, the triangles and the pinned positions. Three settings are optional. Unlike a `Body`, the callbacks are given to the constructor.

Input or function	Meaning
`vertices` and triangle indices	The mesh: corner positions, and three corner numbers per triangle.
`fixedPointsPositions`	Corners that are pinned and do not move. Use the same positions as in `vertices`.
`bendingCompliance` (default 1)	How easily the cloth bends. Bigger means softer. It is not a spring stiffness.
`substeps` (default 4)	How many small steps per physics step. Must be above 0.
`movementSimulation` (default 0.8)	How much of a movement you apply is simulated (chapter 18).
`generateVertices()`	Returns the current corner positions, in the order you gave them.
`applyTransformation(matrix, simulate)`	Moves or turns the cloth.
`addCloth`, `getCloth`, `eraseCloth`, `clearCloths`	Add, find and remove cloth. Cloth is stored apart from bodies.

Corners that are at exactly the same position are joined into one physical point. A 3D model often has such doubled corners at its seams. `generateVertices()` still gives you one position per original corner, so your mesh keeps working. A cloth cannot be copied or moved in memory, because its parts point to each other: always create it with `std::make_shared`.

The constructor checks its input and throws an exception if something is wrong: a missing world, invalid numbers, a triangle index that does not exist, or a mesh with loose points. `addCloth` throws if the cloth is `nullptr` or if its owner is already used.

> **The engine does not draw the cloth**
> It only gives you positions. Your renderer must copy them into the mesh, recalculate the normals if the material needs them, and upload everything to the graphics card.

In the code: modules/VPE.ixx: callback types (line 2868), addCloth (line 2882), class Cloth (line 3340), Cloth constructor (line 3456), generateVertices (line 3534).

Part II - How the engine works inside

Follow one update from start to end.

Click a chapter title to jump to it.

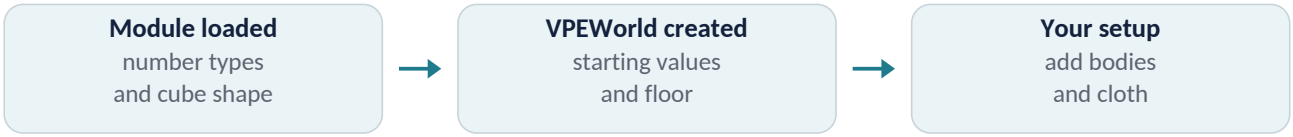
Chapter	Page
9. Files and starting values	14
10. One tick, step by step	15
11. Time steps, leftover time and pausing	16
12. How forces become movement	17
13. Broad phase: finding nearby pairs	18
14. Narrow phase: the exact test	19
15. Solving contacts	20
16. Joints inside the solver	21
17. How cloth is built and moved	22
18. Cloth and bodies push each other	23

Each chapter is one page. Read them in order the first time. Later you can jump straight to the chapter you need.

9. Files and starting values

What happens when you create a world.

Almost the whole engine is in one file, `modules/VPE.ixx`. It contains the classes and also their code. The small file `modules/VPE.cpp` only has two math helper functions.



The `VPEWorld` constructor itself is empty. All starting values come from the member variables. One of them is the floor: a very large, unmovable body with owner `nullptr`.

Starting value	What it means for you
60 steps per second (`m_sim_frequency`)	The engine always moves in steps of 1/60 second.
`m_loops = 30`	Up to 30 rounds to solve contacts and joints in each step.
`m_solver = 0`	Which contact solver is used (chapter 15).
Grid cell width 3 (`m_width`)	Size of the cells used to find nearby bodies (chapter 13).
Floor at y = 0	Always there. A floor you draw does not change it.
No main loop	Nothing happens until your game calls `tick`.

Create the world in a place where it stays, add your callbacks and bodies, and then start calling `tick`. Do not copy or move the world later: every body remembers the world's address.

In the code: modules/VPE.ixx: settings (lines 900-945), floor m_ground (line 1054), VPEWorld constructor (line 2859); modules/VPE.cpp.

10. One tick, step by step

Set a breakpoint in tick and step over these lines.

`tick` first works out how many physics steps are due (chapter 11). For each step it does this:

Order	What happens in one physics step
1	**Broad phase:** quickly find pairs of bodies that are near each other.
2	**Narrow phase:** test those pairs exactly and create or update contacts.
3	**Warm start:** reuse the pushes from the last step for contacts that still exist.
4	Change each body's speed using its forces.
5	Prepare the joints, then repeat up to 30 times: correct the speeds at contacts and joints.
6	Remember the old position, move each body with its new speed, update its matrices.
7	Move every cloth, in several small steps, including its contacts with bodies.
8	Apply the pushes the cloth gave to bodies, and update their matrices.

After all due steps are done, the world updates which grid cell each body is in. Then it calls the move callbacks of all bodies and all cloth. So your callbacks run once per tick, not once per step.

> **Where your own code fits**
> Before `tick`: your game sets forces. During `tick`: the engine calculates and then calls your callbacks, which publish the new positions. After `tick`: your renderer draws. The engine never talks to the graphics card.

In the code: modules/VPE.ixx: tick (line 1234), broadPhase (line 1361), narrowPhase (line 1377), calculateImpulses (line 1573).

11. Time steps, leftover time and pausing

Why the engine moves in fixed steps.

A game's frames are not all equally long. Physics is more stable when every step has the same length. So the engine collects the time you give it and does one fixed step of 1/60 second each time enough time has passed. In simple form:

```
// Simplified version of what tick does:
current_time = last_time + dt;
while (current_time > next_step_time) {
    do_one_step(1/60 second);
    last_step_time = next_step_time;
    next_step_time += 1/60 second;
}
call_callbacks(current_time - last_step_time);   // the leftover time
last_time = current_time;
```

Note the >: if you call `tick(1.0/60.0)` exactly once at the start, the time is equal, not greater, so no step happens yet. The next call then does the step.

If a frame was slow, one tick does several steps to catch up. There is **no limit**. After a long pause or a stop in the debugger, a huge `dt` can make the engine work for a long time. Decide in your game what should happen: make `dt` smaller, skip the paused time, or catch up on purpose.

The move callback gets the **leftover time**: the time since the last step. You can use it to guess where the body is between two steps, so the motion looks smoother. Keep that guess separate from the real physics position.

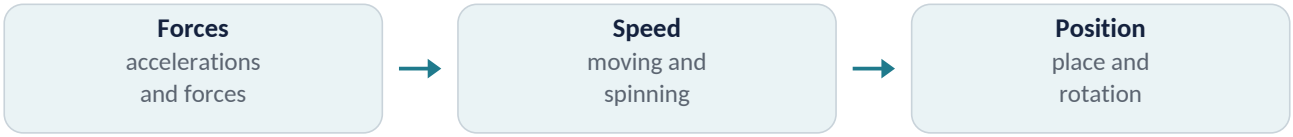
Thing to know	Why
Debug mode	In `SIMULATION_MODE_DEBUG`, `tick` ignores `dt`. The example moves the clock itself and then calls `tick(0)`.
`m_run` does nothing	The variable exists, but `tick` does not check it. To pause, do not call `tick`.
Changing the step rate	Setting `m_sim_frequency` alone does not change `m_sim_delta_time`. Change both.

In the code: modules/VPE.ixx: tick (line 1234), time variables (lines 909-945); examples/physicsexample/physicsexample.cpp, debug stepping (around line 650).

12. How forces become movement

First the speed changes, then the position.

In each step, `Body::stepVelocity` adds up all forces of a body. Accelerations are added directly. Forces are multiplied by the inverse mass (heavy bodies speed up less). A force on a point of the body also makes it spin; the inertia decides how much.



A force stays until you remove or replace it. Gravity, added once, acts in every step. That is good for gravity, but bad for a jump: a jump force that stays would push the body up forever. For a one-time push, change the speed directly instead (chapter 24).

Stored value	Why the engine needs it
Inertia (local, and inverse in world space)	How hard it is to make the body spin in its current rotation.
Previous position and rotation	Needed when cloth touches the body.
Position and speed corrections	Small fixes that push overlapping bodies apart. Not a game force.
Resting counter and damping	Lets bodies that lie still calm down instead of jittering.
Model matrix and its inverse	Turns the shape's corners into world positions for the collision test.

`Body::stepPosition` then moves the position and rotation using the speeds. Note: even when the example calls it on copies to guess a drawing position, it still updates some resting information of the body.

> **One place for the truth**
> Do not move the visible object on its own while the physics body moves somewhere else. Change the physics body, and let the callback copy its position to the visible object.

In the code: modules/VPE.ixx: Body::stepVelocity and stepPosition (from line 575), computeModel (line 680).

13. Broad phase: finding nearby pairs

A quick filter before the exact test.

Testing every body against every other body would be slow. So the engine puts the bodies on a flat grid, seen from above (x and z). Each body is in the cell where its center is. Only bodies in the same or a neighboring cell become a **pair**. Every body is also paired with the floor.

Being a pair does not mean the bodies touch. The broad phase only says "these two might touch". The narrow phase (chapter 14) checks it exactly.

Situation	What to watch out for
Very large bodies	Two big bodies can overlap while their centers are more than one cell apart. Then the pair is missed.
Very fast bodies	The engine checks at each step, not in between. A fast, thin object can pass through another.
Negative coordinates	Cells are computed by cutting off decimals, so cells next to 0 behave a bit differently.
Several steps in one tick	The cell of each body is updated only after all steps of the tick.
Changing the cell width	Bodies keep their old cells. You must sort them in again.

Choose a cell width (`m_width`, default 3) that fits your object sizes. Bigger cells miss fewer pairs, but create more pairs to test.

> **Debugging a missed collision**
> First check whether the two bodies ever became a pair. If not, the problem is the grid or the owners, and changing friction or other contact settings will not help.

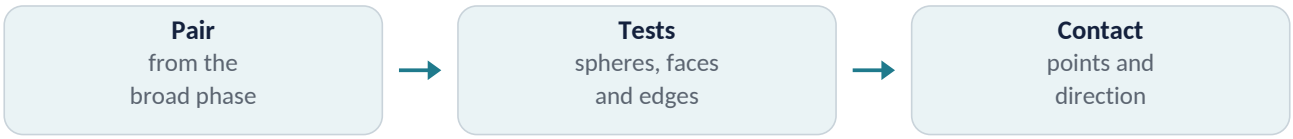
In the code: modules/VPE.ixx: addGrid (line 1089), broadPhase (line 1361).

14. Narrow phase: the exact test

From a possible pair to real touching points.

For each pair, the engine first checks two spheres around the bodies. If the spheres do not touch, the bodies cannot touch either, and the pair is skipped. Otherwise the engine does an exact test called **SAT** (separating axis test).

The idea of SAT: if you can find a direction in which the two shapes do not overlap, they do not collide. The engine tries the directions of all faces and of edge pairs. If none separates the shapes, they collide, and the engine calculates the touching points and how deep they overlap.



The floor is a special case: the engine simply checks which corners of the body are below y = 0.

Contacts are kept from step to step. The engine compares old and new touching points. This lets it reuse the pushes from the last step (warm starting), which makes stacks of boxes stable. Contacts that no longer touch are deleted.

Your collision callback (chapter 25) is called from here when a pair touches. It is not called in every step for bodies that lie still, and there are no separate "started touching" or "stopped touching" events.

> **Do not change the world inside a callback**
> Collision callbacks run while the engine is going through its lists. Only write down what happened, and make your changes after `tick` has returned.

In the code: modules/VPE.ixx: narrowPhase (line 1377), groundTest (line 1478), SatQuery and SAT (from line 1596).

15. Solving contacts

Small pushes that stop bodies from sinking into each other.

When two bodies touch, the engine gives each a short push, an **impulse**, at the touching point. An impulse changes the speed at once, and because it acts away from the center it can also change the spin. Both bodies get a push, in opposite directions. How much each one reacts depends on its mass and inertia.

Setting or idea	What it does
`m_solver = 0`	Default. Solves the push and the friction together.
`m_solver = 1`	Solves the push first and then the friction in two side directions.
Push (normal impulse)	Can only push, never pull, so touching bodies do not stick together.
Friction	Slows sliding. It can be at most the push times the friction value.
Bounciness (restitution)	How much a body bounces back.
`m_bias`, `m_slop`	Push overlapping bodies apart; a tiny overlap (0.001) is allowed.

Warm starting (`m_use_warmstart`, on by default) starts with the pushes of the last step for contacts that still exist. This makes stacks stable with less work.

The engine repeats the solving up to `m_loops` (30) times per step. In normal (real-time) mode it also stops when one step's time (1/60 second) is used up, so a slow computer may do fewer rounds. In debug mode there is no time limit.

> **Same input does not always mean same result**
> Because of the time limit, two computers can get slightly different results. Do not build networked multiplayer that needs exactly the same physics everywhere without testing this first.

In the code: modules/VPE.ixx: settings m_bias, m_slop, m_solver, m_loops (lines 906-918), calculateImpulses (line 1573).

16. Joints inside the solver

A joint is corrected again and again, together with contacts.

Before the solving rounds, each joint prepares itself (`setUp`): it calculates where its connection points are now and how strongly it must correct. In each round, `solveVelocity` pushes the two bodies so the unwanted movement gets smaller. Contacts and joints are solved in the same rounds, so a mechanism and the ground it touches affect each other.

Joint	The error it tries to remove
`DistanceConstraint`	The centers are closer or farther apart than they should be.
`BallSocketJoint`	The two copies of the shared point have drifted apart.
`HingeJoint`	The shared point has drifted, or the bodies turn off the axis. Optional limits and motor.
`FixedJoint`	Any change in distance or rotation between the bodies.
`SliderJoint`	Sideways movement or rotation. Optional limits and motor along the axis.

When you create a joint, the shared point you give in world coordinates is stored in each body's own local coordinates. When the bodies move, the engine turns these local points back into world points and compares them. A limit only becomes active when the movement reaches it.

Joints can shake or stretch when bodies have very different masses, when the connection point is in the wrong place, when contacts fight against the joint, or when there are too few solving rounds.

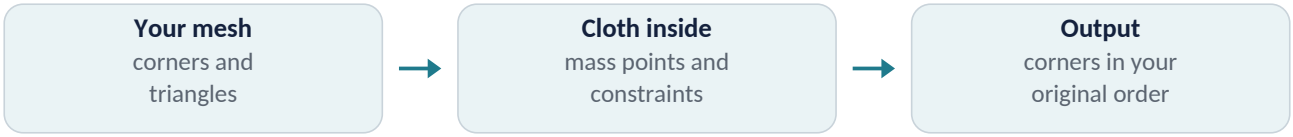
> **A good way to watch a joint in the debugger**
> Set breakpoints in the joint constructor, in `setUp` and in `solveVelocity`. The local points stay the same; the world points, the error and the total push change.

In the code: modules/VPE.ixx: class Constraint (line 1869), HingeJoint (line 2082), setupConstraints and calculateImpulses (line 1573).

17. How cloth is built and moved

From a mesh to a net of points.

When you create a cloth, it first checks the mesh. Then it joins corners at the same position into one **mass point** and remembers which original corners belong to which point. Each triangle edge becomes a **constraint** that keeps two points at their distance. Extra bending constraints connect points across shared edges. Bigger triangles give heavier points.



The pinned positions decide which points stay fixed. The constructor also turns the cloth by a tiny angle, so a perfectly flat cloth can start to move in all directions. So the very first positions are slightly different from your input.

In every physics step, each cloth does its own number of smaller steps (`substeps`). In each small step it slows the points a little (damping), adds gravity, guesses the new positions, corrects the constraints, and handles contacts with bodies and the floor. **Cloth gets gravity automatically**, unlike bodies.

Compliance means softness. The engine divides it by the small step length squared, so it is not the same as a spring stiffness. More substeps make the cloth more exact, but cost more time.

> **Always use generateVertices**
> Give your renderer the output of `generateVertices()`. It has one position per original corner, in your order. The inner list of mass points has fewer entries and a different order.

In the code: modules/VPE.ixx: ClothMassPoint (line 2948), ClothConstraint (line 3206), class Cloth (line 3340), createMassPoints (line 3884).

18. Cloth and bodies push each other

A body reacts to the cloth in the same step.

Cloth looks at the bodies near it. Because it makes several small steps, it blends the body's old and new position for each small step, so it can follow a moving body. This is special for cloth: bodies among themselves do not get this treatment.

When a cloth point touches a body, the overlap is shared by mass: a light point moves more, a heavy body moves less. The push and the friction change the point's speed and also the body's speed and spin.

While the cloth moves	After all cloth has moved (still in the same step)
Collect the pushes for each body	Move and turn the body by the collected amounts.
Collect position corrections	Apply the position corrections.
Use the body's old and new position	Update the body's matrices if it moved.
Solve cloth constraints and contacts	Clear the collected values for the next step.

So a cloth can push or pull a body right away, not one frame later. If you trace a body's final position in the debugger, look after the cloth part of the step.

`applyTransformation(matrix, simulate)` moves a cloth. With `simulate = false` the whole cloth simply jumps. With `true`, pinned and free points move by different amounts (set by `movementSimulation`), so the cloth swings and bends as if it were carried.

> **Cleaning up cloth**
> Cloth has its own list in the world. `clear()` does not remove cloth and does not call its erase callbacks: use `clearCloths()`.

In the code: modules/VPE.ixx: tick, cloth part (line 1265), Cloth::integrate (line 3488), clearCloths (line 2902).

Part III - Building a game with it

Practical recipes, in the order you will need them.

Click a chapter title to jump to it.

Chapter	Page
19. Get the code and build it	25
20. A falling body	26
21. Keeping objects alive	27
22. Connecting the renderer	28
23. Physics in the game loop	29
24. Forces, jumps and direct changes	30
25. Collision events and deleting objects	31
26. Building a mechanism	32
27. Adding cloth	33
28. Speed and stability	34
29. Troubleshooting	35
30. Walking through the demo program	36
31. Quick reference	37

Each chapter is one page. Read them in order the first time. Later you can jump straight to the chapter you need.

19. Get the code and build it

Same steps as in the quickstart.

Install Git and Visual Studio 2022 or newer with *Desktop development with C++* and *C++ CMake tools for Windows*. Then, in PowerShell:

```
cd C:\data\GitHub
git clone https://github.com/hlavacs/ViennaPhysicsEngine.git
cd ViennaPhysicsEngine
.\build_windows.cmd debug --without-vve
```

This builds the physics engine on its own and runs its tests. In your program you write `import VEPhysicsEngine;` and, in CMake, link `ViennaPhysicsEngine::ViennaPhysicsEngine`. Writing `import` alone is not enough: CMake must build the module too (quickstart page 3). The engine needs a compiler with C++23 module support and CMake 3.28 or newer.

CMake option	What it does (default)
`VPE_BUILD_EXAMPLES`	Build the 3D demo, which needs the Vienna Vulkan Engine (ON).
`VPE_BUILD_TESTS`	Build the engine tests (OFF; the script turns it on with `--without-vve`).
`VPE_VVE_IN_TREE`	Build the Vulkan engine together with VPE (ON on Windows).
`VPE_DOUBLE_ACCURACY`	Use `double` for all physics numbers (OFF).
`VPE_FORCE_LEFT_HANDED`	Use a left-handed coordinate system (OFF).

The 3D demo needs more: the Vulkan SDK, vcpkg, CMake 3.31.8 or newer, and the Vienna Vulkan Engine in the folder next to VPE. Then `.\build_windows.cmd debug` builds everything, and the demo is `ViennaVulkanEngine\bin\debug\exe\physicsexample.exe`.

> **Why the Vulkan engine is built too**
> With the Microsoft compiler, C++ modules only fit together when both sides are compiled with exactly the same settings. Building both in one go guarantees that. The demo works with the current Vulkan engine (module `VVEngine`) and with older versions (module `VEEngine`); CMake picks the right one.

In the code: build_windows.cmd; CMakeLists.txt, options at the top; examples/physicsexample/CMakeLists.txt.

20. A falling body

Make a cube, add gravity, let time pass.

```
#include <memory>
import VEPhysicsEngine;

struct GameObject { int id = 1; };

int main() {
    GameObject owner;             // Must not move in memory.
    vpe::VPEWorld world;          // Must live longer than the body.
    using W = vpe::VPEWorld;
    auto body = std::make_shared<W::Body>(
        &world, "falling cube", &owner, &W::g_cube,
        glmvec3{1,1,1}, glmvec3{0,4,0},
        glmquat{1,0,0,0}, glmvec3{0,0,0},
        glmvec3{0,0,0}, 1.0_real, 0.2_real, 0.8_real);
    body->setForce(0ul,
        W::Force{{0, world.c_gravity, 0}});
    world.addBody(body);
    for (int frame = 0; frame < 120; ++frame)
        world.tick(1.0 / 60.0);
    world.eraseBody(body);
}
```

The cube starts 4 units above the floor. Force number 0 is gravity, so it falls. Inverse mass 1 means mass 1. After two seconds (120 ticks of 1/60 second) it lies on the floor with its center at about 0.5.

The owner lives on the stack, so its address stays the same. There is no callback, so nothing is drawn, but the world still moves the body. At the end, `eraseBody(body)` removes it; this version would also remove joints, if there were any.

> **Watch these values in the debugger**
>
> On the body: `m_positionW`, `m_linear_velocityW`, `m_mass_inv` and `m_forces`. On the world: `m_loop`, `m_current_time` and `m_next_slot`. Remember from chapter 11 that the first tick may not do a step yet.

In the code: modules/VPE.ixx: Body constructor (line 535), addBody (line 1099), tick (line 1234), eraseBody (line 1136).

21. Keeping objects alive

Plan who owns what before you add callbacks.

The engine stores the **address** of your game object. If the object moves in memory, the address is wrong. This happens, for example, with a `std::vector<GameObject>` that grows. Use `std::vector<std::unique_ptr<GameObject>>` instead: the pointers move, but the objects stay where they are.

Object	Must stay alive…
`VPEWorld`	as long as any body or cloth uses it.
`Polytope` (shape)	as long as any body uses it.
Your game object (owner)	until its body or cloth is removed and no callback uses it anymore.
Body `shared_ptr`	is shared by the world, your game object, contacts and joints.
Visible object and mesh	as long as move callbacks write into it.
Everything a callback captures	until the callback is removed or can no longer be called.

A `shared_ptr` only keeps the body alive. It does not keep the owner, the shape or anything your callbacks use alive. A good design: each game object stores its body and its visible object together, and a scene class owns the world and the shapes.

Shut down in this order: stop calling `tick`; call `clearCloths()` while the visible cloth still exists; call `clear()` while erase callbacks can still reach your game and renderer; then delete the scene, the shapes and finally the world.

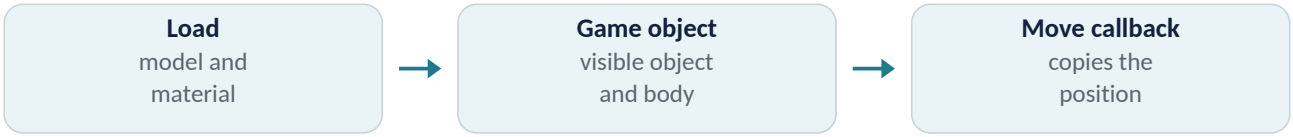
> **clear() is not a full reset**
> `clear()` removes bodies and joints, but not cloth, and it does not reset the clock or the floor. For a completely fresh start, delete the world and create a new one.

In the code: modules/VPE.ixx: clear (line 1119), eraseBody (line 1136), clearCloths (line 2902).

22. Connecting the renderer

Your code copies physics results to the visible objects.

Load 3D models, materials and textures with the Vienna Vulkan Engine or your own loader. Create the collision shape separately. A box model can use `g_cube`. A complex model usually needs a few convex shapes that you choose. Loading a model does **not** create a physics body.




```
// publishModel is YOUR function that moves the visible object.
body->m_on_move = [publishModel](double leftover,
                                 std::shared_ptr<W::Body> b) {
    auto p = b->m_positionW;
    auto q = b->m_orientationLW;
    publishModel(W::Body::computeModel(p, q, b->m_scale));
};
// Also set m_on_erase, to remove the visible object.
```

The simplest start is to copy the physics position as it is, like above. If you later use the leftover time to guess a smoother position, do not write that guess back into the body.

For cloth, copy the output of `generateVertices()` into the mesh, and recalculate the normals if needed. Your renderer copies the data to the graphics card and draws it. VPE never touches the graphics card.

> **The floor you see and the floor that collides**
> A textured plane in the Vulkan engine only shows a floor. The physics floor is always at y = 0. Put your visible floor there, or objects will seem to float or sink.

In the code: modules/VPE.ixx: Body::m_on_move (line 479), computeModel (line 680); examples/physicsexample/physicsexample.cpp, addCube (around line 305).

23. Physics in the game loop

When your game writes, when physics runs, when the screen reads.

```
// One frame of your game. All of these are YOUR functions,
// except world.tick.
pollInput();
updateGameplay(elapsed);
applyQueuedPhysicsCommands();
if (!paused)
    world.tick(elapsed);
processQueuedCollisionEvents();
processQueuedObjectRemoval();
updateCameraAndUI(elapsed);
drawScene();
```

The order matters. Game rules set forces **before** the physics step. The callbacks publish new positions **during** `tick`. Adding and removing objects happens **after** `tick`. Then the scene is drawn.

If the Vulkan engine already runs the main loop, call `tick` from your update function there, with the elapsed time it gives you in seconds. Do not start a second loop.

Situation	What to do
Pause and continue	Do not call `tick` while paused. Do not pass the paused time afterwards.
Stepping in the debugger	Pass a small fixed time, or use debug mode (chapter 11).
A very slow frame	Decide on a maximum `elapsed` value, for example 0.1 seconds.
Changing the step rate	Change `m_sim_frequency` and `m_sim_delta_time` together.
Several threads	Only one thread may use the world at a time. The world is not thread-safe.

> **Measure before you change settings**
> Watch the frame time, how many steps each tick does, the solver time and the number of bodies and cloths. A slow frame causes more catch-up steps, which can make the next frame slow too.

In the code: modules/VPE.ixx: tick (line 1234).

24. Forces, jumps and direct changes

Steady pushes, one-time pushes and editing values by hand.

Steady pushes like gravity, an engine or wind are **forces**. Give each kind its own ID number, for example 0 for gravity, 1 for the player's engine and 2 for wind, so they do not overwrite each other.

```
W::Force thrust;
thrust.m_accelW = glmvec3{3, 0, 0};
body->setForce(1ul, std::move(thrust));
// When the player lets go of the key:
body->removeForce(1ul);
```

A jump or a hit is a **one-time push**. Do not use a force for it, because a force stays. Instead, change the speed directly, for example `body->m_linear_velocityW.y += 5;`. Then check that the body really starts moving: there is no separate "wake up" function.

If you change body values by hand, the engine's stored values may no longer fit:

You changed...	Then also...
position or rotation	call `updateMatrices()`, and move the body to its new grid cell if it moved far.
size or inverse mass	call `inertiaTensorL()` and `updateMatrices()`.
the position by a lot (teleport)	remove and re-add the body, so the grid and contacts are correct.
the shape	keep the old and new shape alive, and recalculate inertia and matrices.

> **Easier: remove and create again**
> For a reset or a respawn, it is often easier to remove the body and create a new one than to change many values. Do it outside `tick`, and add the forces, callbacks and joints again.

In the code: modules/VPE.ixx: struct Force (line 268), setForce and removeForce (line 556), updateMatrices (line 687).

25. Collision events and deleting objects

Turn engine callbacks into safe game events.

Register a collision callback for a body with `addCollider(body, callback)`. When that body touches something, the callback gets two bodies: first your body, then the other one. Use their `m_owner` to find your game objects. The other body may be the floor, whose owner is `nullptr`.

```
// Only write the event down here; handle it after tick.
world.addCollider(body,
    [&events](std::shared_ptr<W::Body> mine,
              std::shared_ptr<W::Body> other) {
        events.push_back({mine, other});
    });
```

`events` is your own list, for example a `std::vector` of pairs. It keeps the bodies alive, but not their owners, so empty the list before you delete game objects.

The callback can be called in many steps in a row for the same pair, and not in every step for objects that lie still. If you need "started touching" and "stopped touching", keep your own list of touching pairs and compare it from frame to frame.

Step	Why
1. In the callback: only write down the event	The engine is in the middle of its lists.
2. Let `tick` finish	Contacts and cloth must be completely done.
3. Call `eraseBody(body)`	This also removes the body's joints.
4. Remove the visible object and the owner	The erase callback must be able to finish its work first.
5. Throw away old events	So no later code uses a deleted owner.

In the code: modules/VPE.ixx: addCollider (line 1162), narrowPhase calls the callbacks (line 1400), eraseBody (line 1136).

26. Building a mechanism

Check the connection points and axes on screen.

Start with two bodies and one joint: an unmovable support (inverse mass 0) and one moving part. Give the connection point in world coordinates at the place where the parts should meet. Hinges and sliders also need an axis that points the right way and is not zero.

```
auto hinge = std::make_shared<W::HingeJoint>(
    support, door, hingePoint, glmvec3{0,1,0});
hinge->enableLimit(-1.0_real, 1.0_real);   // radians
hinge->enableMotor(0.5_real, 2.0_real);    // speed, maximum force
world.addConstraint(hinge);
// Stop the motor but keep the hinge:
hinge->disableMotor();
// The door breaks off:
world.removeConstraint(hinge);
```

For a hinge, the motor speed is a turning speed; for a slider it is a sliding speed. The maximum force is a limit for the solver, not an exact model of a real motor. Read the comments in the joint's code before you use real motor values.

Draw small debug markers at both connection points and a line along the axis. Check that the movement looks right before you add more parts. Build chains and rag dolls one joint at a time. Very different masses, limits that fight each other, and shapes that block the movement all cause problems.

> **When it shakes**
> Check the connection point, the axis direction, overlapping shapes at the start, the mass ratio, the time step and the number of solving rounds before you make the correction stronger. A stronger correction can hide the problem for a while but makes things more unstable.

In the code: modules/VPE.ixx: HingeJoint (line 2082), enableLimit (line 2203), enableMotor (line 2229), SliderJoint (line 2506).

27. Adding cloth

Keep the physics cloth and the visible mesh connected.

```
// vertices, triangleIndices and pinnedPositions come from your mesh loader.
auto cloth = std::make_shared<W::Cloth>(
    &world, "banner", &bannerObject,
    onClothMove, onClothErase,
    vertices, triangleIndices, pinnedPositions,
    0.1_real, 8, 0.8_real);
world.addCloth(cloth);
// In onClothMove(leftover, clothPtr):
//     publishPositions(clothPtr->generateVertices());
// To remove it:
world.eraseCloth(&bannerObject);
```

Types: `vertices` and `pinnedPositions` are `std::vector<glmvec3>`, `triangleIndices` is `std::vector<uint32_t>`. The move callback gets a `double` and a `std::shared_ptr<W::Cloth>`; the erase callback gets only the `shared_ptr`. `publishPositions` is your own function.

Pinned positions must be exactly on corners of the mesh. Corners at the same position are joined, so check the seams of your model. The constructor throws an exception for bad input, so create cloth while loading, not in the middle of a frame.

Setting	What to test
Substeps	Better contacts against more CPU time.
Bending compliance	How much the cloth folds and bends.
Damping	How fast the cloth calms down.
Friction and bounciness	How cloth slides on and bounces off bodies.
Movement simulation	How free points follow when you move the cloth with `applyTransformation`.

Start with a small mesh with a pinned top edge. Check gravity, the corner order and cleaning up first. Add contacts with bodies and moving the cloth later. The cloth does not collide with itself.

In the code: modules/VPE.ixx: class Cloth (line 3340), Cloth constructor (line 3456), addCloth (line 2882), eraseCloth (line 2914).

28. Speed and stability

Change one thing at a time, and measure.

If you…	you get…	but…
raise the step rate	smaller, more exact steps	more steps per second cost more CPU time.
raise the solving rounds	steadier contacts and joints	each step takes longer.
have a heavy scene in real-time mode	the time limit stops solving early	results get less exact.
use bigger grid cells	fewer missed pairs	more pairs to test.
use more cloth substeps	better cloth contacts	more CPU time.
use a finer cloth mesh	smoother cloth	more points, constraints and contacts.
keep warm starting on	stable stacks with less work	check that resting stacks really stay still.

Measure normal gameplay and the worst case. A test with only falling cubes does not tell you how a stack, a chain of joints or a cloth on a moving body behaves. Count pairs and contacts, not only bodies: a few bodies that overlap a lot can be slower than many bodies that are far apart.

Many values belong together: step rate and step time, mass and inertia, position and matrices and grid cell, cloth softness and substeps. Do not change one of them without the others. Save the settings together with your test scenes, so you can repeat a test later.

> **A small set of test scenes**
> Keep five scenes: a falling cube, a stack of boxes, a box on a slope, a hinge or slider, and a cloth touching a body. Write down positions, speeds and joint errors over time, not only whether the program still runs.

In the code: modules/VPE.ixx: settings (lines 900-945), calculateImpulses (line 1573); tests/vpe_compile_check.cpp.

29. Troubleshooting

Find the first place where the values go wrong.

Problem	Check first
A body does not fall	Inverse mass above 0? Gravity added? Does `tick` really do steps?
The visible object does not move	Is a move callback set? Does it move the right visible object?
A body falls through another	Did they become a pair (grid)? Too fast or too thin? Convex shape?
Objects float or sink into the floor	The physics floor is at y = 0. Move the visible floor there. Check units.
Joints stretch or shake	Connection point, axis, overlap, mass ratio, time step, correction, solving rounds.
Cloth constructor throws	Valid numbers? Triangle indices exist? Three per triangle? Substeps above 0?
Cloth tears open at seams	Use `generateVertices()` in the original order and check the mesh indices.
Crash after deleting	Owner, shape or callback data deleted too early? Removed with `eraseBody(owner)` while joints still exist?
Long freeze after a pause	A huge `dt` makes the engine catch up step by step.
Picking selects strange bodies	`pickBody` picks the body most in the look direction; it is not an exact ray test.

`pickBody` also crashes if there are no bodies at all, so check first. For exact picking, write your own ray test and then find the body by its owner.

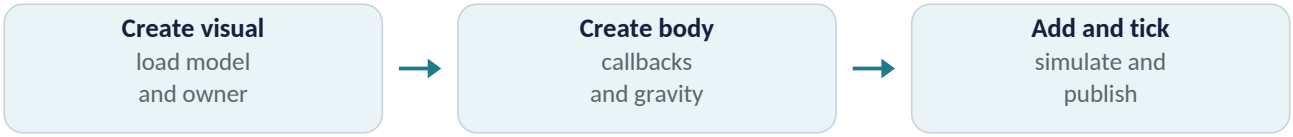
Print the values before and after the step where it goes wrong. If two bodies never become a pair, changing the friction cannot help. If the physics is right but the picture is not, more solving rounds cannot help: the problem is in the connection to the renderer.

In the code: modules/VPE.ixx: pickBody (line 1078), addGrid (line 1089), tick (line 1234).

30. Walking through the demo program

Follow one object from creation to the screen.

Open `examples/physicsexample/physicsexample.cpp`. Start where the world and the Vulkan engine are created. Then find the function `addCube` (around line 305), which creates a body. It makes a `shared_ptr` with the world, a name, the game owner, the cube shape, size, position and material values. Then it sets the move and erase callbacks and adds gravity.



Set a breakpoint in `addBody` and look at `m_bodies` and the grid. In `tick`, notice the difference between one frame and one physics step. Step into `broadPhase`, `narrowPhase` and `calculateImpulses` only for the body you care about. Watch its speed change first, then its position.

In the move callback, follow the owner back to the visible object. The callback calculates the matrix and hands it to the Vulkan engine, which draws it later. The physics engine stops at the callback.

For cloth, find the `make_shared<VPEWorld::Cloth>` call (around line 269), then `addCloth`. Step into `integrate` and compare the generated positions with the order of the loaded mesh.

> **The demo works with old and new Vulkan engines**
> The demo imports `VVEngine` (the current Vulkan engine) or `VEEngine` (older versions). CMake checks which one your Vulkan engine provides and sets `VPE_VVE_USES_VVENGINE`. The physics module is always `VEPhysicsEngine`.

In the code: examples/physicsexample/physicsexample.cpp: imports (line 25), cloth (line 269), bodies (line 314); examples/physicsexample/CMakeLists.txt.

31. Quick reference

The most important functions on one page.

Task	Use
Create the world	`import VEPhysicsEngine;` then `vpe::VPEWorld world;`
Create a solid object	`std::make_shared<VPEWorld::Body>(...)` with a shape and an owner
Add or remove it	`addBody(body)`, `eraseBody(body)`
Push it	`setForce(id, force)`, `removeForce(id)`
React to movement	`m_on_move`, `m_on_erase`
React to collisions	`addCollider(body, callback)`
Connect two bodies	a joint in a `shared_ptr`, then `addConstraint` / `removeConstraint`
Create cloth	`std::make_shared<VPEWorld::Cloth>(...)`, `addCloth`, `generateVertices`
Move time forward	`tick(seconds)`
Clean up	`clearCloths()` and `clear()`, in that order

Next steps: start with the falling cube from chapter 20. Then add, one at a time: a renderer (chapter 22), the game loop (23), collision events (25), joints (26) and cloth (27). The test file `tests/vpe_compile_check.cpp` contains more small examples of bodies, joints and cloth.

> **What a finished game integration has**
> Owners and shapes that stay alive, only convex collision shapes, a clear game loop, callbacks that copy positions to the screen, changes made only after `tick`, a clean shutdown, and a few test scenes you can repeat.

In the code: modules/VPE.ixx; tests/vpe_compile_check.cpp.

Where to find things in the code

Open these places in your editor.

What	File and line
Module name and number types	modules/VPE.ixx, line 98
`VPEWorld` and its settings	modules/VPE.ixx, line 237 (settings from line 900)
`Force` and shape parts	modules/VPE.ixx, line 268
`Body`	modules/VPE.ixx, line 461
Contact records	modules/VPE.ixx, line 716
`MapWrapper` (body storage)	modules/VPE.ixx, line 954
`addBody`, `clear`, `eraseBody`	modules/VPE.ixx, lines 1099-1160
`tick`	modules/VPE.ixx, line 1234
`broadPhase`, `narrowPhase`	modules/VPE.ixx, lines 1361 and 1377
`calculateImpulses` (solver)	modules/VPE.ixx, line 1573
SAT collision test	modules/VPE.ixx, line 1596
Joints	modules/VPE.ixx, from line 1869 (hinge 2082, fixed 2425, slider 2506)
Cloth functions in the world	modules/VPE.ixx, lines 2866-2935
Cloth parts	modules/VPE.ixx, lines 2948 (mass point) and 3206 (constraint)
`Cloth`	modules/VPE.ixx, line 3340
Two math helpers	modules/VPE.cpp, line 6
Demo: bodies and cloth	examples/physicsexample/physicsexample.cpp, lines 314 and 269
Build options	CMakeLists.txt, top of the file
Tests and small examples	tests/vpe_compile_check.cpp

Line numbers are for 12 October 2026 (commit f14aec8). If the code has changed, search for the name instead.